LAB University of Applied Sciences

EV Charging Data Analyzer

Markdown and HTML documentation

LAB University of Applied Sciences
Industrial Information Technology 24S
2026.
Parth Mangukiya

Contents

1 Project and README ..... **Error! Bookmark not defined.**
2 Markdown syntax 1
3 How Markdown appears in HTML 3
3.1 Project files and code formatting 4
3.2 Bullet points bold text and tables 5
3.3 File paths in code blocks 6
4 Complete Markdown file 7
5 Self Reflection 13
6 Reference 13
7 AI Reflection 13

1 Project and README

For this exercise, I using a new project called the EV Charging Data Analyzer. I did not reuse a project from an earlier course. This report explains its README and shows how the Markdown appears in HTML. The complete Markdown text is included in Section 4.

The programme reads electric vehicle charging data from a CSV file. It calculates averages and creates three charts. It uses two external Python libraries: pandas for working with the data and Matplotlib for making charts. Parth Mangukiya maintains the project.

The README explains the purpose, requirements, installation and use of the programme. It also gives example results and the name of the maintainer. The HTML file can be opened locally in a web browser, so a public website is not needed.

2 Markdown syntax

Markdown uses simple symbols to organise and format text. The table below gives examples from the README and explains how they appear in HTML.

Markdown example	Purpose	Appearance in HTML
# Heading	Main heading	A large heading at the top of the document.
## Heading	Section heading	A smaller heading for a main section.
### Heading	Subheading	A heading within a section.
text	Bold text	The text appears thicker, so it stands out.
- item	Bullet list	Each item has a bullet beside it.
1. item	Numbered list	Each item has a number beside it.

`requirements.txt`	Inline code	The file name appears in a font with equal character widths.

``bash
python --version
``
	Fenced code block	The command appears in a separate block. Spaces and line breaks are kept.

| Name | Value | | --- | ---: |	Table	Text appears in rows and columns. The colon aligns the second column to the right.
---	Dividing line	A horizontal line appears between sections when the hyphens are on a separate line.

Markdown gives the page its structure. CSS controls its appearance, such as colours, borders and spacing. For example, the blue background behind a file name comes from CSS, not from the backticks alone.

The screenshots in Section 3 show these elements in the HTML page. The full README.md text is included in Section 4.

3 How Markdown appears in HTML

The following screenshots show the README in a web browser. Each screenshot gives an example of how Markdown text becomes part of the HTML page.

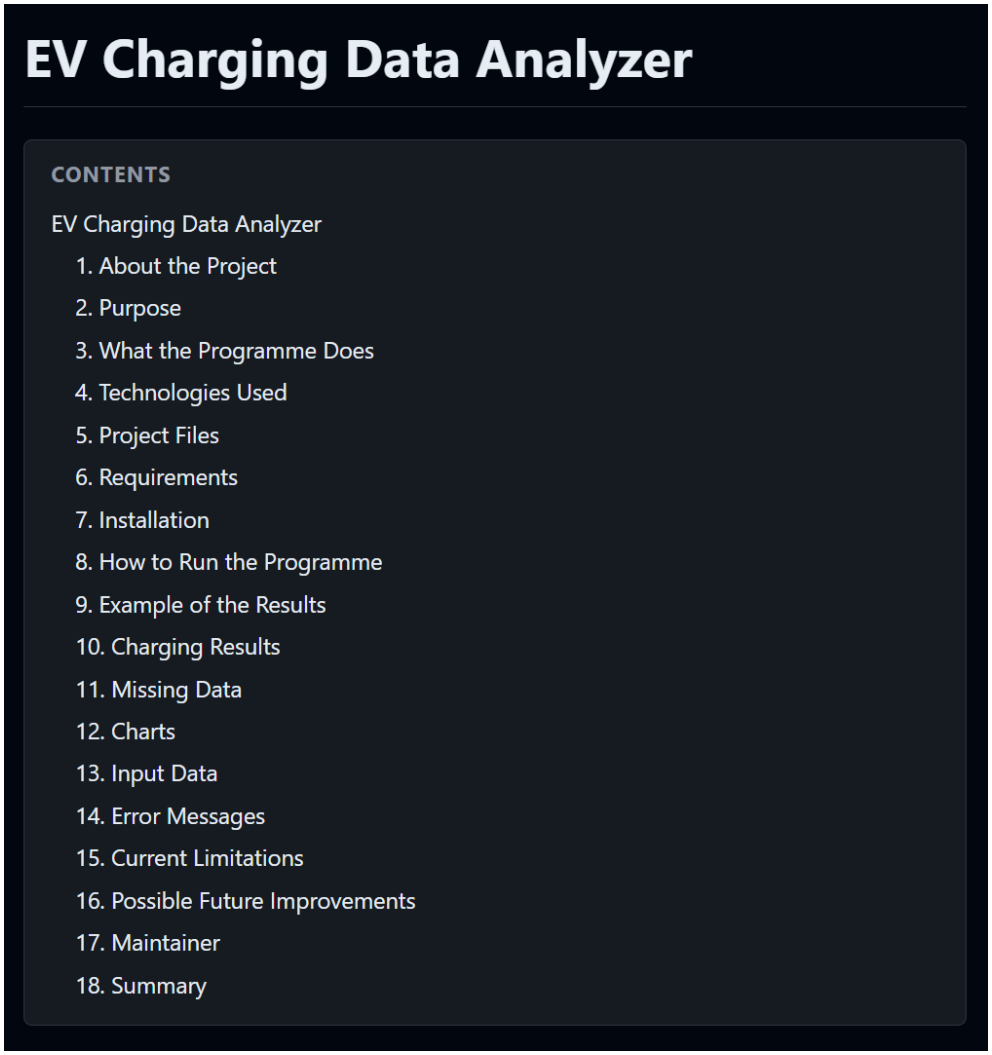
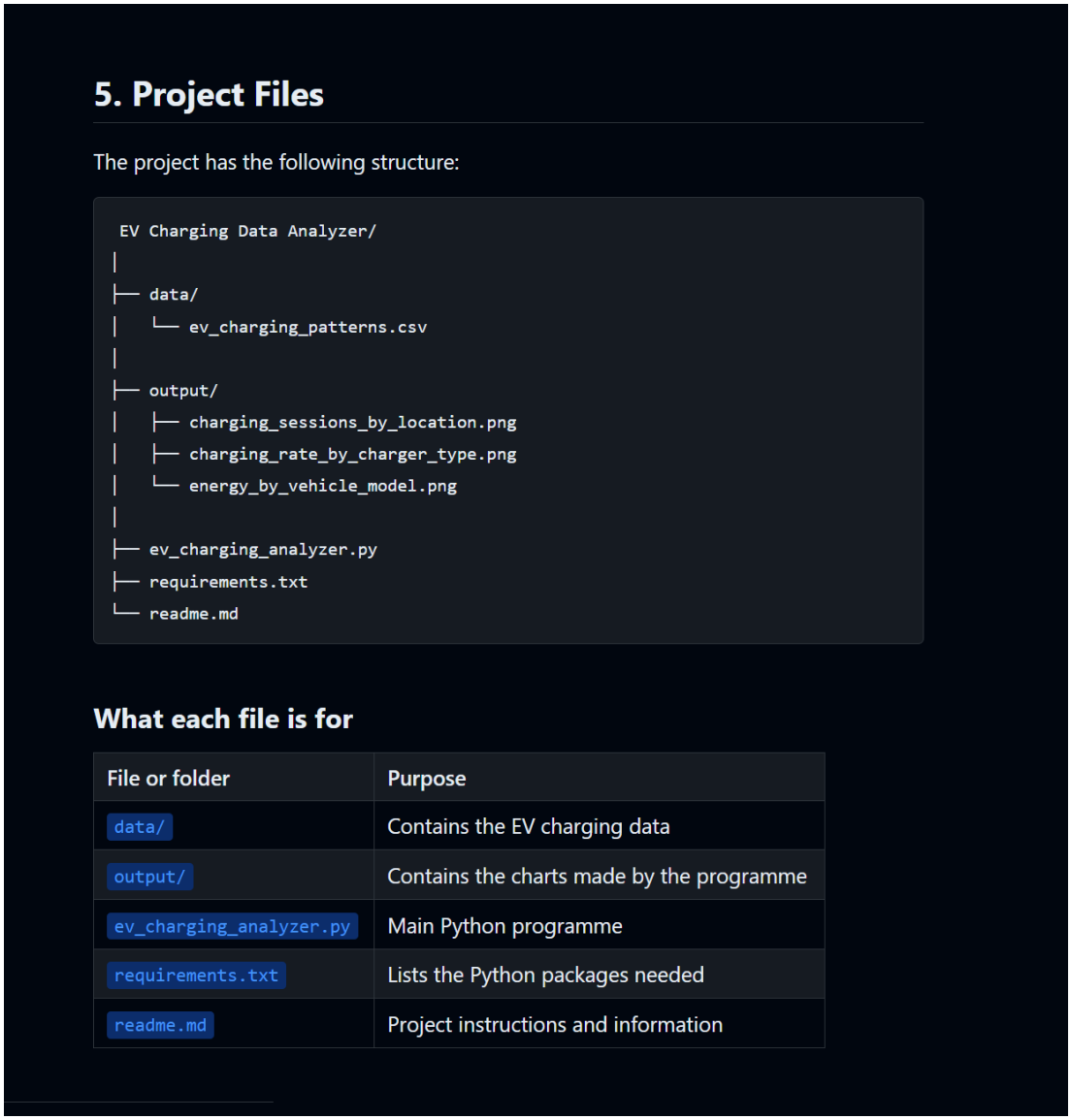


Figure 1. Main heading and contents menu

The main heading appears as large, bold text. In the Markdown source, one hash sign (#) marks this heading. The contents menu lists the sections and provides links to them. This menu is added during HTML conversion; it is not a numbered list in the README source. The dark colours and the border around the menu come from CSS.

3.1 Project files and code formatting



5. Project Files

The project has the following structure:

```
EV Charging Data Analyzer/
│
├── data/
│   └── ev_charging_patterns.csv
│
├── output/
│   ├── charging_sessions_by_location.png
│   ├── charging_rate_by_charger_type.png
│   └── energy_by_vehicle_model.png
│
├── ev_charging_analyzer.py
├── requirements.txt
└── readme.md
```

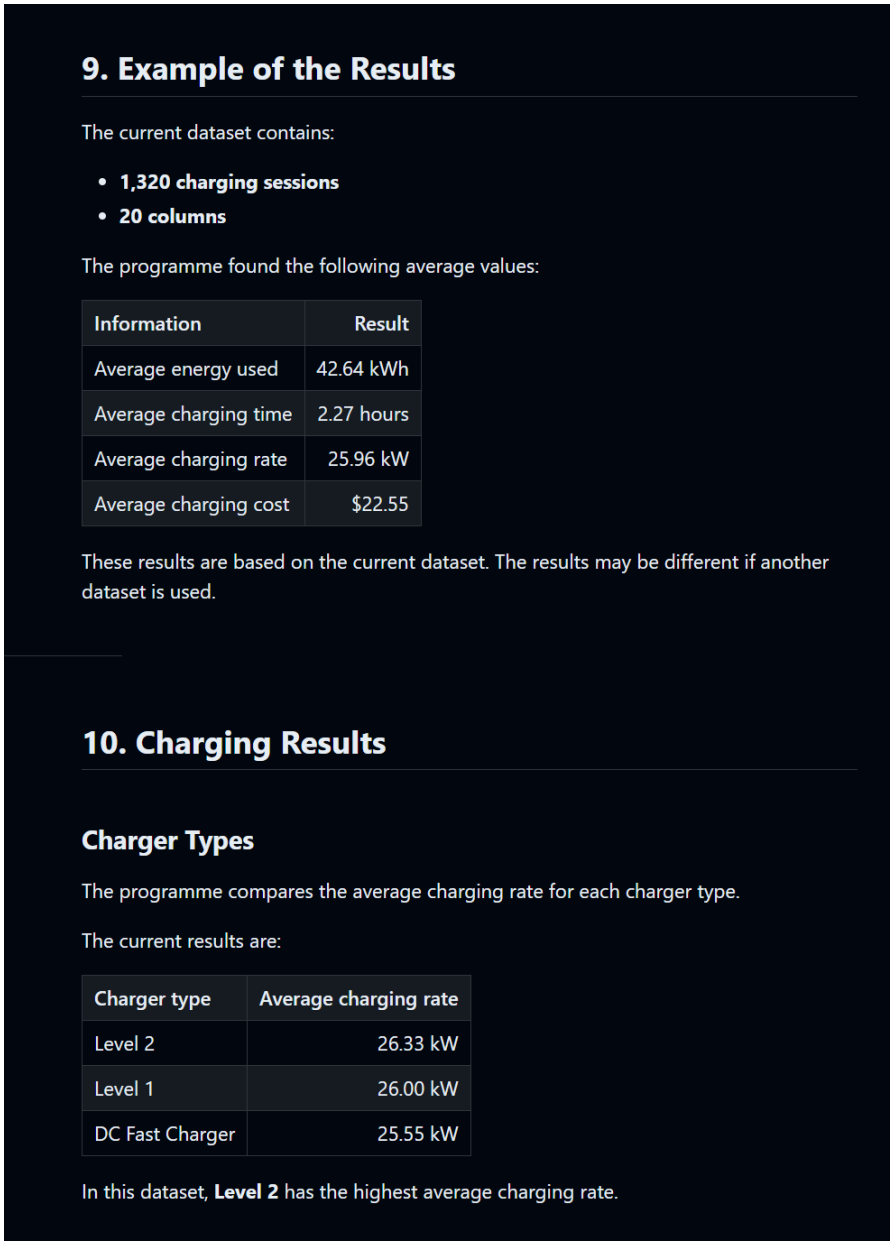
What each file is for

File or folder	Purpose
`data/`	Contains the EV charging data
`output/`	Contains the charts made by the programme
`ev_charging_analyzer.py`	Main Python programme
`requirements.txt`	Lists the Python packages needed
`readme.md`	Project instructions and information

Figure 2. A code block, a table and inline code

The folder structure is inside a fenced code block, marked by three backticks in Markdown. In HTML, it keeps its spaces and line breaks and uses letters with equal widths. The file descriptions appear in a table with rows and columns. Single backticks mark file names as inline code. CSS gives these names a blue background and adds the table borders.

3.2 Bullet points bold text and tables



9. Example of the Results

The current dataset contains:

- **1,320 charging sessions**
- **20 columns**

The programme found the following average values:

Information	Result
Average energy used	42.64 kWh
Average charging time	2.27 hours
Average charging rate	25.96 kW
Average charging cost	$22.55

These results are based on the current dataset. The results may be different if another dataset is used.

10. Charging Results

Charger Types

The programme compares the average charging rate for each charger type.

The current results are:

Charger type	Average charging rate
Level 2	26.33 kW
Level 1	26.00 kW
DC Fast Charger	25.55 kW

In this dataset, **Level 2** has the highest average charging rate.

Figure 3. Example results and charger types

The two lines below the first heading appear as bullet points because they start with hyphens in Markdown. Two asterisks on each side make the text bold. Vertical bars and a header separator form the tables. A colon at the right end of a separator aligns the numbers to the right. The heading sizes show the section levels, and a horizontal line separates sections.

3.3 File paths in code blocks

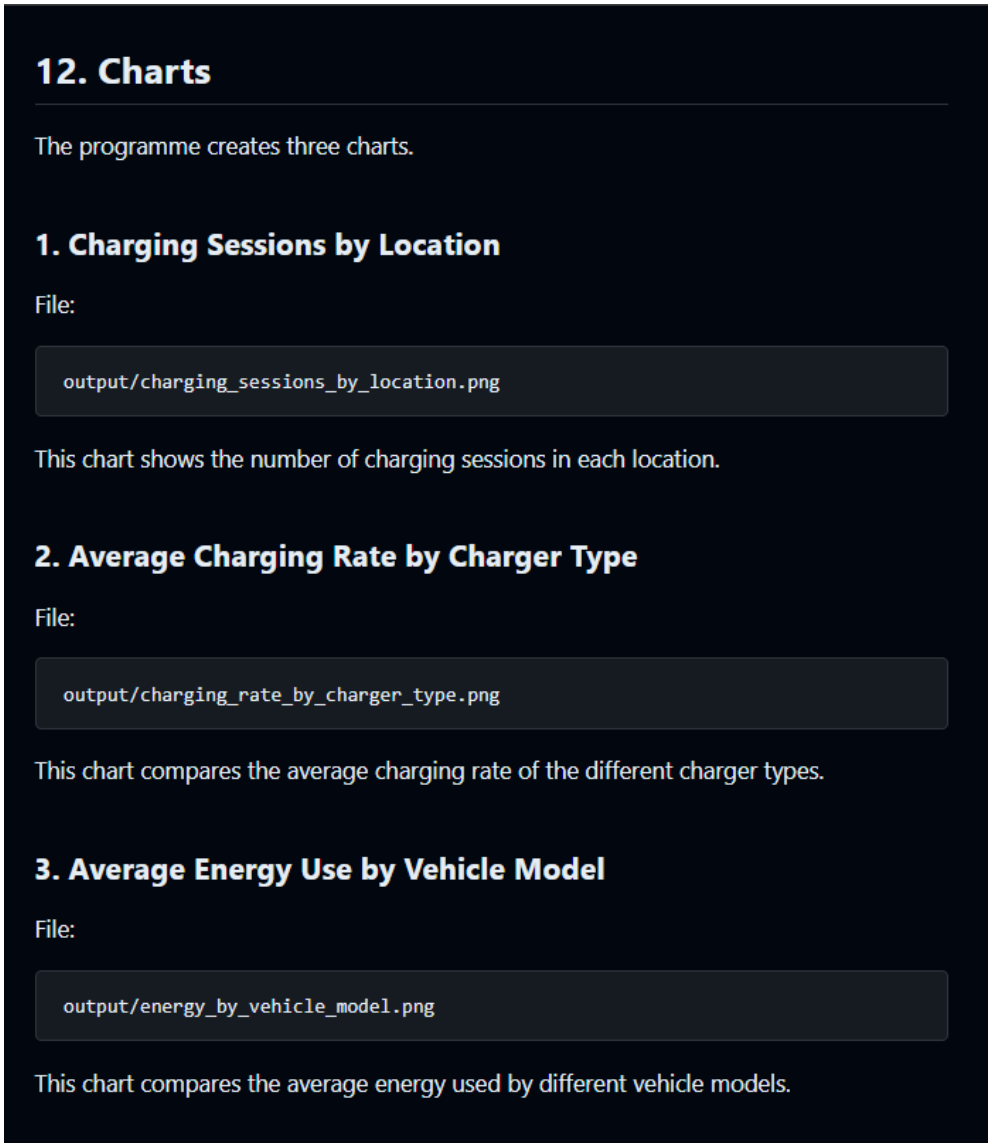


Figure 4. Chart file paths shown as code blocks

The chart file paths appear in separate code blocks because they are enclosed by three backticks in Markdown. The letters have equal widths, so the paths are easy to read. CSS adds the darker boxes and their borders. The numbered chart titles are headings written with three hash signs (###); they are not items in a numbered Markdown list. This part of the page shows file paths and descriptions, not the chart images themselves.

4 Complete Markdown file

The text below is the complete README.md file. The Markdown symbols are kept so that they can be compared with the HTML page.

```
# EV Charging Data Analyzer

## 1. About the Project

The **EV Charging Data Analyzer** is a Python programme that looks at electric vehicle (EV) charging data.

The programme reads data from a CSV file and gives simple information about EV charging. It also creates charts to make the results easier to understand.

The programme can show:

- How many charging records are in the file
- The names of the data columns
- Missing data
- Average energy used
- Average charging time
- Average charging speed
- Average charging cost
- Charging sessions by location
- Charging speed for different charger types
- Energy used by different car models

---

## 2. Purpose

The main purpose of this project is to make EV charging data easier to understand.

For example, the programme can help answer questions such as:

- How much energy is used during an average charging session?
- How long does charging normally take?
- Which charger type has the highest average charging speed?
- Which car model uses the most energy?
- Which location has the most charging sessions?

The programme also creates charts so that the results can be seen more easily.

---

## 3. What the Programme Does

When the programme starts, it reads the EV charging data from a CSV file.

It then:

1. Loads the data.
2. Shows the number of rows and columns.
3. Shows the names of the columns.
4. Checks for missing data.
5. Calculates average charging information.
6. Compares different charger types.
7. Compares different vehicle models.
8. Counts charging sessions by location.
9. Creates three charts.
10. Saves the charts in the `output` folder.

---
```

4. Technologies Used

The programme is written in **Python**.

It uses two external Python packages:

Pandas

Pandas is used to read the CSV file and work with the data.

It helps the programme:

- Read the data
- Find missing values
- Calculate averages
- Group information
- Count records

Matplotlib

Matplotlib is used to create the charts.

The programme uses it to create and save the following charts:

- Charging sessions by location
- Average charging rate by charger type
- Average energy use by vehicle model

---

5. Project Files

The project has the following structure:

```text
EV Charging Data Analyzer/
│
├── data/
│   └── ev_charging_patterns.csv
│
├── output/
│   ├── charging_sessions_by_location.png
│   ├── charging_rate_by_charger_type.png
│   └── energy_by_vehicle_model.png
│
├── ev_charging_analyzer.py
├── requirements.txt
└── readme.md
```

What each file is for

File or folder	Purpose
`data/`	Contains the EV charging data
`output/`	Contains the charts made by the programme
`ev_charging_analyzer.py`	Main Python programme
`requirements.txt`	Lists the Python packages needed
`readme.md`	Project instructions and information

---

6. Requirements

Before running the programme, you need:

- Python 3
- pip
- The EV charging CSV file

You can check if Python is installed by running:

```bash
python --version
```

You can check pip by running:

```bash
pip --version
```

---

7. Installation

Step 1: Open the project folder

Open a terminal or Command Prompt and go to the project folder.

For example:

```bash
cd "EV Charging Data Analyzer"
```

Step 2: Install the required packages

Run:

```bash
python -m pip install -r requirements.txt
```

The file installs:

```text
pandas
matplotlib
```

Step 3: Check the data file

If the downloaded project contains a folder named `Data`, rename it to `data`. The folder name must match the lowercase path used by the Python programme, especially on case-sensitive systems.

Make sure the CSV file is in:

```text
data/ev_charging_patterns.csv
```

---

8. How to Run the Programme

After installing everything, run:

```bash
python ev_charging_analyzer.py
```

The programme will start reading the data and show the results in the terminal.

After the analysis is finished, the charts will be saved in the `output` folder.

---

9. Example of the Results

The current dataset contains:

- **1,320 charging sessions**

- **20 columns**

The programme found the following average values:

Information	Result
Average energy used	42.64 kWh
Average charging time	2.27 hours
Average charging rate	25.96 kW
Average charging cost	\$22.55

These results are based on the current dataset. The results may be different if another dataset is used.

---

10. Charging Results

Charger Types

The programme compares the average charging rate for each charger type.

The current results are:

Charger type	Average charging rate
Level 2	26.33 kW
Level 1	26.00 kW
DC Fast Charger	25.55 kW

In this dataset, **Level 2** has the highest average charging rate.

Vehicle Models

The programme also compares the average energy used by each vehicle model.

The current results are:

Vehicle model	Average energy used
Tesla Model 3	44.52 kWh
Hyundai Kona	42.83 kWh
Nissan Leaf	42.51 kWh
BMW i3	42.45 kWh
Chevy Bolt	40.74 kWh

In this dataset, the **Tesla Model 3** has the highest average energy use.

Charging Locations

The programme counts the number of charging sessions at each location.

The current results are:

Location	Charging sessions
Los Angeles	297
San Francisco	264
Houston	262
New York	255
Chicago	242

In this dataset, **Los Angeles** has the highest number of charging sessions.

---

11. Missing Data

The programme also checks for missing information.

In the current dataset, there are 66 missing values in each of these columns:

- Energy Consumed (kWh)
- Charging Rate (kW)
- Distance Driven (since last charge) (km)

The programme reports these missing values but does not automatically fill them.

---

12. Charts

The programme creates three charts.

1. Charging Sessions by Location

File:

```text
output/charging_sessions_by_location.png
```

This chart shows the number of charging sessions in each location.

2. Average Charging Rate by Charger Type

File:

```text
output/charging_rate_by_charger_type.png
```

This chart compares the average charging rate of the different charger types.

3. Average Energy Use by Vehicle Model

File:

```text
output/energy_by_vehicle_model.png
```

This chart compares the average energy used by different vehicle models.

---

13. Input Data

The programme uses the following data columns:

```text
User ID
Vehicle Model
Battery Capacity (kWh)
Charging Station ID
Charging Station Location
Charging Start Time
Charging End Time
Energy Consumed (kWh)
Charging Duration (hours)
Charging Rate (kW)
Charging Cost (USD)
Time of Day
Day of Week
State of Charge (Start %)
State of Charge (End %)
Distance Driven (since last charge) (km)
Temperature (°C)
```

Vehicle Age (years)
Charger Type
User Type
```

---

## 14. Error Messages

If the CSV file cannot be found, the programme shows an error message.

For example:

```text
Error: File not found: data/ev_charging_patterns.csv
```

The programme also checks that the data needed for each analysis is available before doing that analysis.

---

15. Current Limitations

The current version has some simple limitations:

- It only reads CSV files.
- The data file must be in the `data` folder.
- The programme runs in the terminal.
- It does not have a graphical interface.
- It does not automatically fix missing data.
- It provides basic analysis and charts.

---

16. Possible Future Improvements

The programme could be improved in the future by adding:

- More charts
- More data analysis
- Automatic handling of missing data
- Analysis of charging times
- Analysis of charging costs
- Temperature analysis
- An interactive dashboard
- Export of results to Excel
- A simple web interface
- Machine learning to predict charging demand

---

17. Maintainer

Maintainer: **Parth Mangukiya**

Project: EV Charging Data Analyzer

This project was created for educational purposes.

---

18. Summary

The **EV Charging Data Analyzer** is a simple Python programme for working with EV charging data.

It reads charging data from a CSV file, finds useful information, calculates averages, compares vehicles and chargers, counts charging sessions by location, and creates charts.

The project shows how Python can be used to turn a large set of raw data into information that is easier to understand.
```

5 Reflection

Writing the README helped me think about someone using my project for the first time. I learnt that I should explain what the project does, which libraries it needs and how to run it. Clear steps make the project easier to use.

I also learnt how Markdown works. For example, # creates a main heading and ## creates a smaller heading. Backticks show file names and commands. Lists make the steps easier to read.

Changing the Markdown file into HTML helped me see how the text appears on a web page. This task showed me why a clear README is useful.

6 References

- **https://www.kaggle.com/datasets/mexwell/electric-vehicle-charging-dataset**

7 AI Reflection

I used AI to help organise the README and improve the wording of my report. It helped me explain the purpose of the project, the libraries it needs, how to install them and how to run the programme. It also helped me use simple words and clear steps so that another person could follow the instructions.

AI helped me explain the Markdown symbols used in the README. These included headings, bold text, lists, tables and code blocks. It also helped me describe how these elements look on the HTML page and explain the screenshots in the report.

The project details came from the EV Charging Data Analyzer files. These included the Python code and the pandas and matplotlib libraries. AI used this information to help write the report. It also helped prepare the Word and PDF versions and improve their layout.

This task helped me see how AI can support writing and explaining technical work. However, AI can make mistakes, so I still need to check the commands, file names and explanations against my project. I also need to make sure that I understand the report before I submit it.